TikTok Trends Dashboard

Technical Design Document

Version 1.0 · March 29, 2026 · Portfolio Project

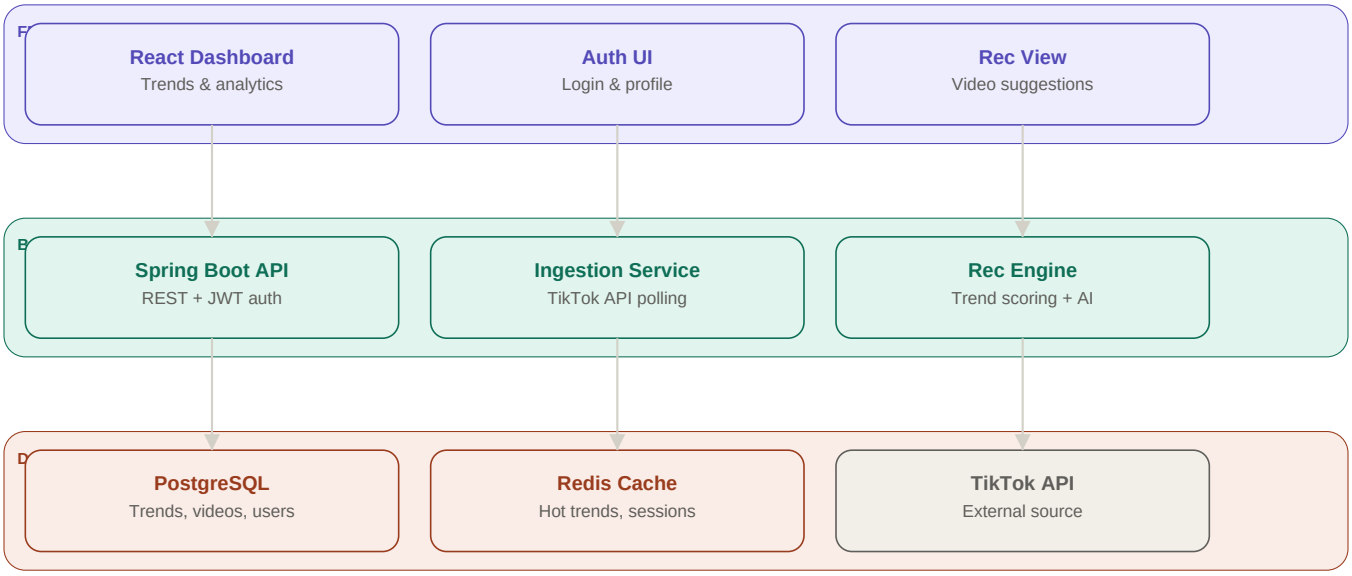
This document describes the end-to-end technical architecture of the TikTok Trends Dashboard, a full-stack application that analyzes real-time TikTok trends and generates AI-powered video recommendations for content creators. It covers system architecture, data flow, database schema, and technology choices.

Tech Stack

Layer	Technology	Rationale
Frontend	React + JavaScript	Existing team strength; component-based UI
Backend	Java 21 + Spring Boot	Industry standard REST + scheduling support
Database	PostgreSQL	Relational model ideal for trend aggregations
Cache	Redis	Sub-millisecond reads for hot trend data
AI	Claude API (Anthropic)	Generates video concepts from scored trends
Migration	Flyway	Version-controlled SQL schema management
Auth	JWT + Spring Security	Stateless auth for React ↔ API communication
Data src	TikTok Research API	Official API — no scraping risk

System Architecture

The system is organized into three tiers: a React frontend consumed by TikTok influencers, a Java Spring Boot backend handling business logic and AI orchestration, and a data layer combining PostgreSQL for persistence and Redis for caching.



Component Descriptions

React Dashboard

The main influencer-facing UI. Displays trending sounds, hashtags, content categories, and engagement velocity charts. Fetches data from the Spring Boot REST API on page load and on manual refresh.

Spring Boot API

The central REST backend exposing versioned endpoints under /api/v1/. Handles JWT-based authentication, request validation, and orchestrates calls between the ingestion service, analytics processor, and recommendation engine.

Ingestion Service

A scheduled Spring component (@Scheduled) that polls the TikTok Research API every two hours. Normalizes and deduplicates raw video records before persisting them to PostgreSQL.

Recommendation Engine

Scores pre-aggregated trends by velocity and engagement rate, then calls the Claude API with a structured prompt containing the top trends and the influencer's niche. Returns a tailored video concept including title ideas, music suggestions, and hashtag sets.

PostgreSQL

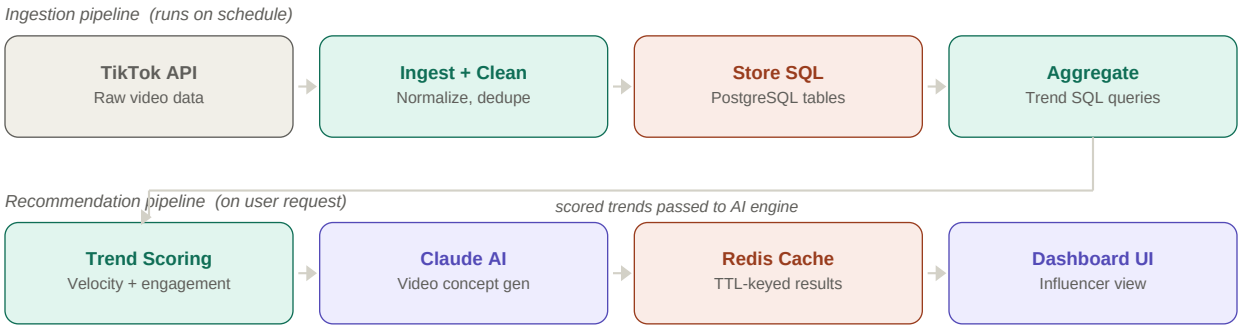
Primary data store. Holds five core tables: users, videos, video_metrics, trends, and recommendations. Schema is managed via Flyway migrations, enabling versioned, reproducible deployments.

Redis Cache

Caches the results of expensive trend aggregation queries with a one-hour TTL. Prevents redundant database hits when multiple users view the dashboard simultaneously.

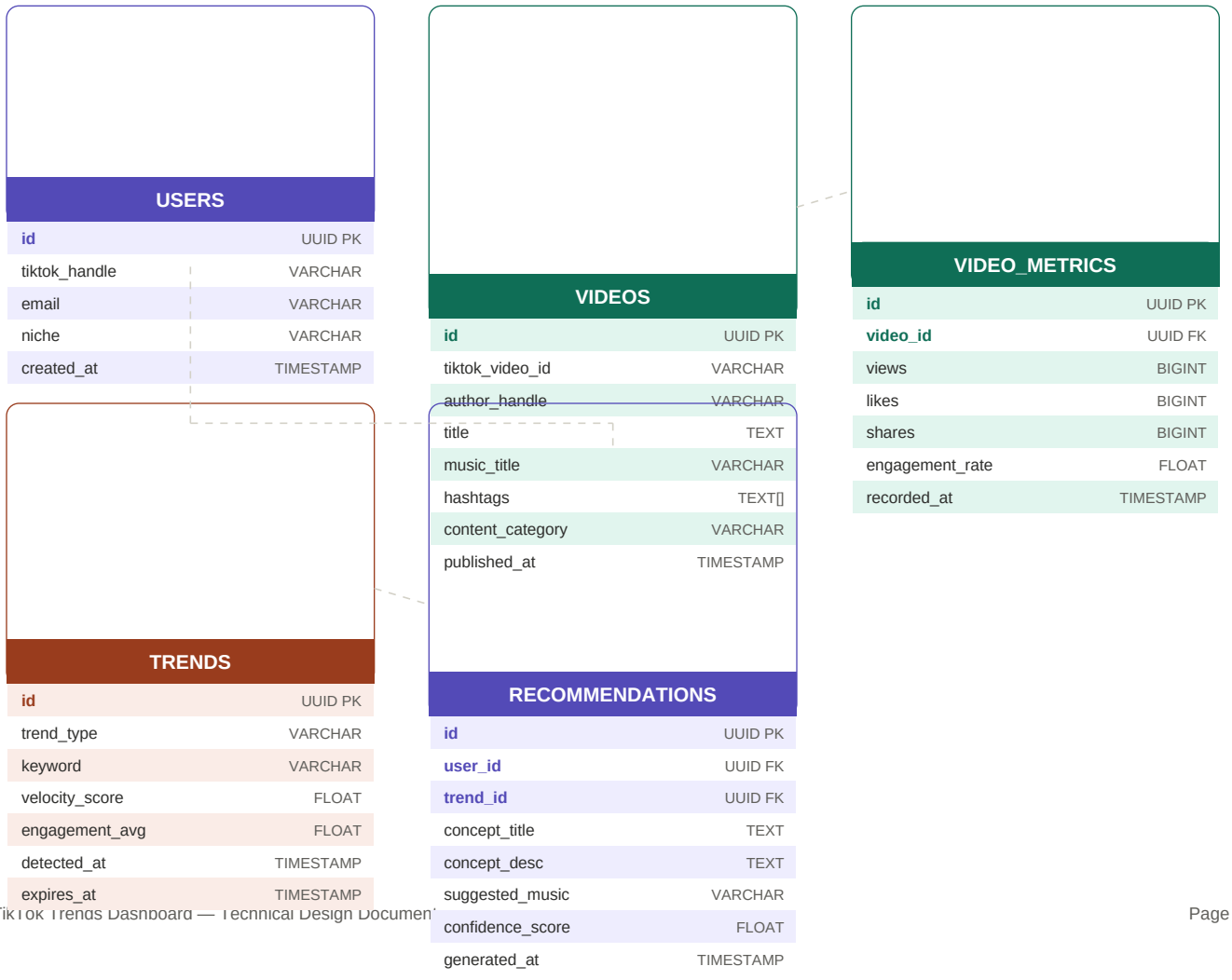
Data Flow

The system operates two independent pipelines. The ingestion pipeline runs on a CRON schedule and pre-computes trend data. The recommendation pipeline runs on demand when an influencer requests their next video idea, consuming the pre-computed trends.



Database Schema (ERD)

Five normalized tables store all application data. Foreign key relationships link VIDEO_METRICS to VIDEOS, and RECOMMENDATIONS to both USERS and TRENDS. Dashed lines indicate FK relationships.



Key API Endpoints

Method	Endpoint	Description
GET	/api/v1/trends	List current trends sorted by velocity score
GET	/api/v1/trends/music	Top trending music tracks with engagement stats
GET	/api/v1/trends/hashtags	Top hashtags by category and region
POST	/api/v1/recommendations	Generate AI video concept for authenticated user
GET	/api/v1/recommendations/{id}	Fetch a previously generated recommendation
POST	/api/v1/auth/login	Authenticate user, returns JWT
POST	/api/v1/auth/register	Register new influencer account
GET	/api/v1/videos/trending	Recent high-velocity video feed
POST	/api/v1/ingest/trigger	Admin: manually trigger ingestion run

MVP Build Order

Phase 1	**Foundation**	PostgreSQL schema via Flyway · Spring Boot project skeleton · JWT auth endpoints
Phase 2	**Data Ingestion**	TikTok Research API client · Ingestion service · CRON scheduling
Phase 3	**Trend Engine**	SQL aggregation queries · Velocity scoring logic · Redis caching layer
Phase 4	**AI Recs**	Claude API integration · Prompt engineering · Recommendations endpoint
Phase 5	**Frontend**	React dashboard · Trend charts · Recommendation display · Auth flow
Phase 6	**Polish**	Error handling · Loading states · Unit + integration tests · README